5.1 간단한 회로 구현 분석

개요

이 문서는 `examples/simple_circuit.rs` 에서 구현된 PLONK 회로의 구조와 동작 방식을 분석합니다. 이 예제는 PLONK 라이브러리를 사용하여 영지식 증명 회로를 구현하는 기본적인 패턴을 보여줍니다.

회로 구조 분석

TestCircuit 구조체

```
#[derive(derivative::Derivative)]

#[derivative(Debug(bound = ""), Default(bound = ""))]

pub  struct TestCircuit<F, P>

where

F: PrimeField,

P: TEModelParameters<BaseField = F>,

{

a: F, // 첫 번째 입력값

b: F, // 두 번째 입력값

c: F, // a + b의 결과 (공개 입력)

d: F, // a * b의 결과 (공개 입력)

e: P::ScalarField, // 스칼라 곱셈을 위한 스칼라값
```

```
f: GroupAffine<P>, // 스칼라 곱셈 결과 점 (공개 입력)
}
```

회로가 검증하는 제약 조건들

이 회로는 다음 5가지 제약 조건을 검증합니다:

1. **덧셈 제약**: `a + b = c` (c는 공개 입력)
2. **범위 제약 1**: `a ≤ 2^6` (a는 6비트 이하)
3. **범위 제약 2**: `b ≤ 2^4` (b는 4비트 이하)
4. **곱셈 제약**: `a * b = d` (d는 공개 입력)
5. **타원곡선 스칼라 곱셈**: `GENERATOR * e = f` (f는 공개 입력)

회로 구현 방식 분석

1. Circuit 트레이트 구현

```
impl<F, P> Circuit<F, P> for TestCircuit<F, P>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
{
const CIRCUIT_ID: [u8; 32] = [0xff; 32];
fn  gadget(&mut  self, composer: &mut StandardComposer<F, P>) -> Result<(), Erro
// 회로 로직 구현
```

```
}

fn  padded_circuit_size(&self) -> usize {

1 << 9  // 512개의 제약 조건을 위한 크기

}

}
```

2. 변수 추가 및 초기화

```
let  a = composer.add_input(self.a); // 비공개 입력 a 추가

let  b = composer.add_input(self.b); // 비공개 입력 b 추가

let  zero = composer.zero_var(); // 영 변수 생성
```

핵심 개념:

- `add_input()` : 회로에 비공개 입력(witness) 변수를 추가
- `zero_var()` : 영값을 나타내는 특별한 변수 생성
- 각 변수는 `Variable` 타입으로 래핑되어 회로 내에서 참조됨

3. 산술 게이트를 통한 제약 조건 구현

덧셈 제약 (a + b = c)

```
composer.arithmetic_gate(|gate| {

gate.witness(a, b, Some(zero)) // 좌측: a, 우측: b, 출력: zero

.add(F::one(), F::one()) // 계수: 1*a + 1*b

.pi(-self.c) // 공개 입력: -c
```

```
});
```

동작 원리:

- `witness(a, b, Some(zero))` : 게이트의 입력 와이어 설정
- `.add(F::one(), F::one())` : $1a + 1b$ 형태의 선형 결합
- `.pi(-self.c)` : 공개 입력으로 -c 추가하여 a + b - c = 0 제약 생성

곱셈 제약 (a * b = d)

```
composer.arithmetic_gate(|gate| {
gate.witness(a, b, Some(zero)) // 좌측: a, 우측: b, 출력: zero
.mul(F::one()) // 곱셈: 1 * (a * b)
.pi(-self.d) // 공개 입력: -d
});
```

동작 원리:

- `.mul(F::one())` : a와 b의 곱셈 연산 수행
- 결과적으로 a * b - d = 0 제약 생성

4. 범위 게이트를 통한 범위 제약

```
composer.range_gate(a, 6); // a ≤ 2^6 제약
composer.range_gate(b, 4); // b ≤ 2^4 제약
```

동작 원리:

- 범위 게이트는 내부적으로 변수를 이진 분해하여 각 비트가 0 또는 1인지 검증

- 지정된 비트 수만큼의 이진 표현으로 변수를 제한

5. 타원곡선 스칼라 곱셈 제약

```
let  e = composer.add_input(from_embedded_curve_scalar::<F, P>(self.e));

let (x, y) = P::AFFINE_GENERATOR_COEFFS;

let  generator = GroupAffine::new(x, y);

let  scalar_mul_result = composer.fixed_base_scalar_mul(e, generator);

composer.assert_equal_public_point(scalar_mul_result, self.f);
```

동작 원리:

- `from_embedded_curve_scalar` : 임베디드 곡선의 스칼라를 베이스 필드로 변환
- `fixed_base_scalar_mul` : 고정된 베이스 점에 대한 스칼라 곱셈 수행
- `assert_equal_public_point` : 결과 점이 공개 입력과 일치하는지 검증

회로 구성 요소 분석

1. 변수 관리 시스템

- **Variable 타입**: 회로 내 변수들의 추상적 표현
- **witness 변수**: 증명자만 알고 있는 비공개 입력
- **public input**: 증명자와 검증자 모두 알고 있는 공개 입력
- **zero_var**: 특별한 영 변수로 다양한 게이트에서 활용

2. 게이트 시스템

- **arithmetic_gate**: 선형 결합과 곱셈을 지원하는 범용 게이트

- **range_gate**: 변수의 범위를 제한하는 특수 게이트
- **fixed_base_scalar_mul**: 타원곡선 스칼라 곱셈을 위한 특수 게이트

3. 제약 조건 표현 방식

모든 제약 조건은 다음 형태로 표현됩니다:

```
q_L * a + q_R * b + q_O * c + q_M * a * b + q_C = 0
```

여기서:

- `q_L, q_R, q_O, q_M, q_C` : 게이트별 계수들
- `a, b, c` : 와이어 값들
- 공개 입력은 `q_C` 항에 포함됨

회로 크기 및 성능 고려사항

패딩된 회로 크기

```
fn padded_circuit_size(&self) -> usize {
1 << 9  // 512개의 제약 조건
}
```

의미:

- 회로는 2의 거듭제곱 크기로 패딩됨 (FFT 효율성을 위해)
- 실제 제약 조건 수보다 큰 크기로 설정하여 여유 공간 확보
- 크기가 클수록 증명 생성 시간이 증가하지만 더 복잡한 회로 지원 가능

게이트 효율성

1. **산술 게이트**: 하나의 게이트로 선형 결합과 곱셈을 동시에 처리
2. **범위 게이트**: 여러 개의 불린 제약을 하나의 게이트로 압축
3. **스칼라 곱셈 게이트**: 복잡한 타원곡선 연산을 효율적으로 처리

예제에서 사용된 구체적 값들

```
TestCircuit {
a: BlsScalar::from(20u64), // a = 20
b: BlsScalar::from(5u64), // b = 5
c: BlsScalar::from(25u64), // c = 25 (20 + 5)
d: BlsScalar::from(100u64), // d = 100 (20 * 5)
e: JubJubScalar::from(2u64), // e = 2
f: point_f_pi, // f = GENERATOR * 2
}
```

검증 과정:

1. `20 + 5 = 25` ✓
2. `20 ≤ 2^6 = 64` ✓
3. `5 ≤ 2^4 = 16` ✓
4. `20 * 5 = 100` ✓
5. `GENERATOR * 2 = f` ✓

요구사항 충족 분석

요구사항 5.1 충족도

- ✅ **간단한 회로 구현 방법 이해**: TestCircuit 구조체와 Circuit 트레이트 구현 분석 완료
- ✅ **회로 구성 요소 파악**: 변수, 게이트, 제약 조건 시스템 분석 완료

요구사항 5.4 충족도

- ✅ **다양한 제약 조건 타입 사용법 이해**: 산술, 범위, 타원곡선 제약 조건 분석 완료

결론

`simple_circuit.rs` 예제는 PLONK 회로 구현의 핵심 패턴을 잘 보여줍니다:

1. **구조화된 접근**: Circuit 트레이트를 통한 표준화된 인터페이스
2. **다양한 제약 타입**: 산술, 범위, 타원곡선 연산을 모두 지원
3. **효율적인 게이트 활용**: 하나의 게이트로 여러 연산을 동시에 처리
4. **공개/비공개 입력 분리**: witness와 public input의 명확한 구분
5. **확장 가능한 설계**: 새로운 제약 조건을 쉽게 추가할 수 있는 구조

이러한 분석을 통해 PLONK 회로의 기본적인 구현 방식과 설계 철학을 이해할 수 있습니다.